

Searching → In linear data structures.

Linear Search

Binary Search

↓
find element by
checking every possible
value one by one.

↓
find elements in
ordered set of data.

↓
find elements in
unordered set of data.

Linear Search

0	1	2	3
5	1	9	7

$n = 4$

```
boolean linearSearch (int [] arr, int element) {  
    for (int i = 0; i < arr.length; ++i) {  
        if (arr[i] == element) {  
            return true;  
        }  
    }  
    return false;  
}
```

linear

Time Complexity
= $O(n)$

Space complexity
= $O(1)$
↑
Constant

Binary Search

→ Require data to be sorted / arranged in a well defined order.

0	1	2	3	4	5	6	7	8	9
1	5	8	10	12	15	20	30	35	40

$n = 10$



low → 0

high → 9

$$\text{mid} \rightarrow \frac{(\text{low} + \text{high})}{2} = \frac{0 + 9}{2} = 4$$

binary Search(8)

if element < arr[mid] then

element can be found before mid element
(set high end of range to before mid)

0	1	2	3	4	5	6	7	8	9
1	5	8	10	12	15	20	30	35	40

$n = 10$

low $\rightarrow$ 0 ↑ mid

high $\rightarrow$ 3

mid $\rightarrow \frac{0+3}{2} = 1$

if $arr[mid] < element$ then
 element can be found after mid element
 (set low end of range to after mid)

0	1	2	3	4	5	6	7	8	9
1	5	8	10	12	15	20	30	35	40

$n = 10$

low $\rightarrow$ 2

high $\rightarrow$ 3

$$\text{mid} \rightarrow \frac{2+3}{2} = 2$$

mid

if (element ==
arr[mid])

$\Downarrow$

FOUND

Binary Search (18)

0	1	2	3	4	5	6	7	8	9
1	5	8	10	12	15	20	30	35	40

$n = 10$

low $\rightarrow$ ~~0~~ ~~5~~ **6**
high $\rightarrow$ ~~9~~ ~~6~~ **5**
mid $\rightarrow$ ~~4~~ ~~7~~ ~~5~~ 6

mid

mid

high < low
⇓
NOT FOUND

binarySearch(arr, value)

- low = 0;

- high = n - 1

- while (low <= high)

- mid = (low + high) / 2

n : number of element.

$$\rightarrow \text{mid} = \text{low} + (\text{high} - \text{low}) / 2$$

- if (arr[mid] = value)

- Element found

- Stop

- if (value < arr[mid])

// element can be found before mid element, set high end of range to before mid

- high = mid - 1

else

// element can be found after mid element, set low end of range to after mid

- low = mid + 1

- Element not found

logarithmic

Time complexity
 $= O(\log_2 n)$

Space complexity
 $= O(1)$

↑ constant

Sorting



Arranging data in a specific order



Ascending

Smallest to
largest



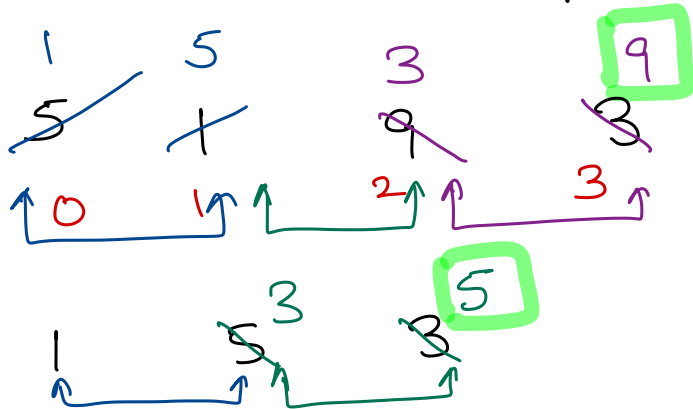
Descending

largest to
smallest

Bubble Sort

Ascending order $\Rightarrow$ in each pair of adjacent elements, we put smaller element to left of larger element.

$\Rightarrow$ if (right element $<$ left element) then
 $\rightarrow$ swap them.



$n = 4$
elementsToSort = 4
left $\rightarrow \cancel{0} \neq 3$



1 3 5 9 ← Sorted elements.

bubbleSort(arr)

- elementsToSort = n
- while (elementsToSort > 1)
 - left = 0
 - while (left < (elementsToSort - 1))
 - if (arr[left + 1] < arr[left])
 - // element on right is less then element on left => they are in wrong positions so swap them
 - swap values at left and (left + 1)
 - Move left ahead by one position
 - Reduce elementsToSort by 1

<u>Outer loop</u> (elements To Sort)	<u>Inner loop</u> (left)	<u># of times inner loop runs</u>
n	0 1 ... (n-2)	= n-1
(n-1)	0 1 ... (n-3)	= n-2
⋮	⋮	⋮
3	0 1	= 2
2	0	= 1
		<u>Total = $\frac{(n-1)(n)}{2}$</u>

$$\frac{(n-1)(n)}{2} = \cancel{\frac{1}{2} (n^2 - n)}$$

Time complexity = $O(n^2)$ Space complexity = $O(1)$

Place correct
element here.
↓

Selection Sort

$n = 4$

0	1	2	3
5 1	1 5	9	7

element Pos $\rightarrow 0$

Smallest Element Pos $\rightarrow$ ~~0~~ 1

$i \rightarrow$ ~~1~~ 2 3

swap
values at these
positions.

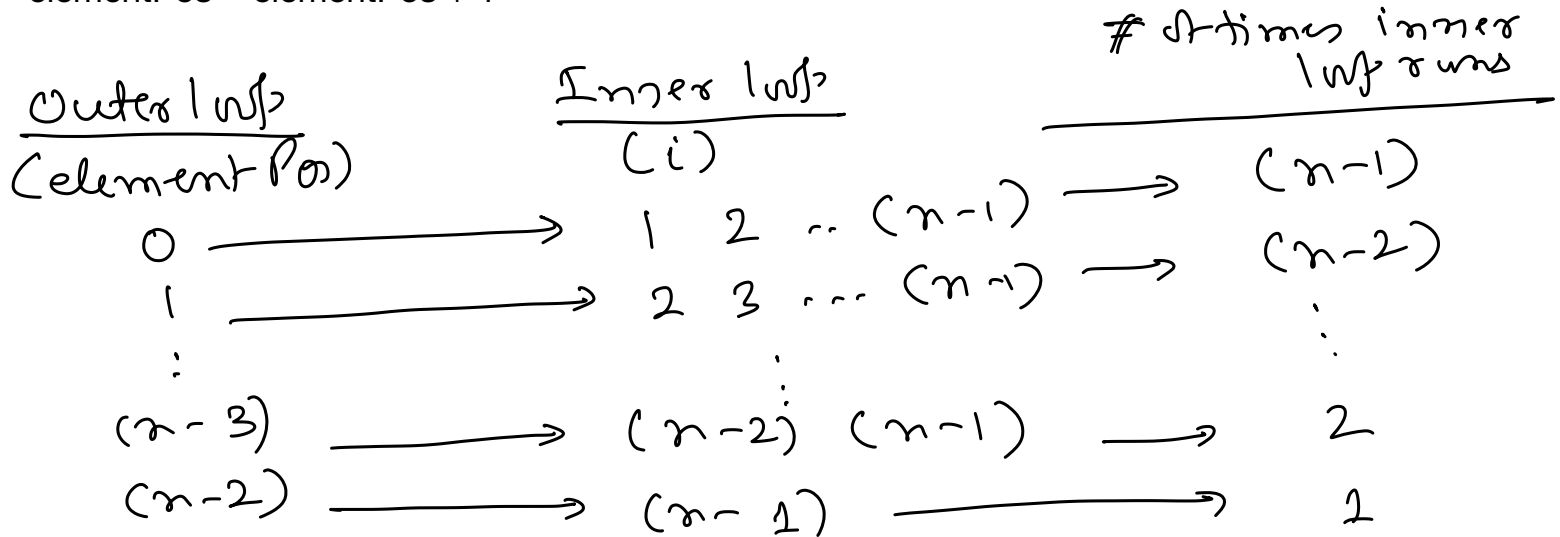
Element Pos $\rightarrow$ ~~0~~ 1

Smallest Element Pos $\rightarrow$ 1

$i \rightarrow$ ~~2~~ 3

selectionSort(arr)

- elementPos = 0
- while (elementPos < (n - 1))
 - smallestElementPos = elementPos
 - i = elementPos + 1
 - while (i < n)
 - if (arr[i] < arr[smallestElementPos])
 - smallestElementPos = i
 - i = i + 1
 - if (elementPos != smallestElementPos)
 - swap values at elementPos and smallestElementPos
- elementPos = elementPos + 1



$$\text{Total} = \frac{(n-1)(n)}{2}$$

$$\frac{(n-1)(n)}{2} = \cancel{\frac{1}{2}} (\cancel{n^2} - \cancel{n})$$

Time complexity = $O(n^2)$

Space complexity = $O(1)$

Bubble Sort



swap Count = 0

Best Case and Worst Case

Data is already sorted.

Data is sorted in reverse order

	<u>Time</u>		<u>Space</u>	
	Best	Worst	Best	Worst
Bubble Sort	$O(n^2)$	$O(n^2)$	$O(1)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(1)$	$O(1)$
Optimised bubblesort	$O(n)$	$O(n^2)$	$O(1)$	$O(1)$

bubbleSort(arr) → Optimised

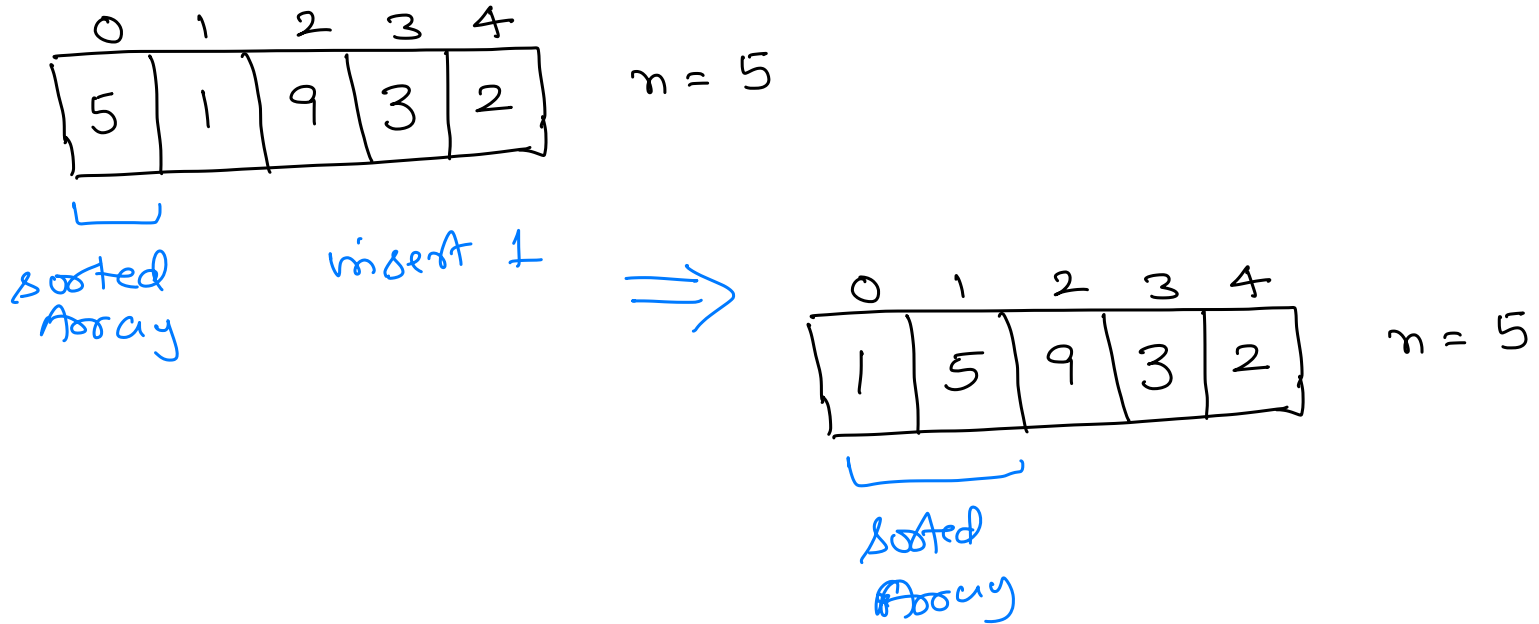
- elementsToSort = n
- while (elementsToSort > 1)
 - left = 0 *Swap Count = 0*
 - while (left < (elementsToSort - 1))
 - if (arr[left + 1] < arr[left])
 - // element on right is less then element on left => they are in wrong positions so swap them
 - swap values at left and (left + 1)
 - Move left ahead by one position
 - Reduce elementsToSort by 1

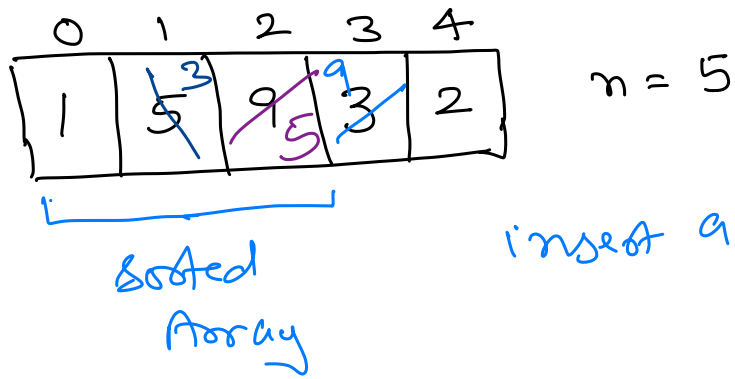
Increment swap count by 1

if (swap count = 0) then stop.

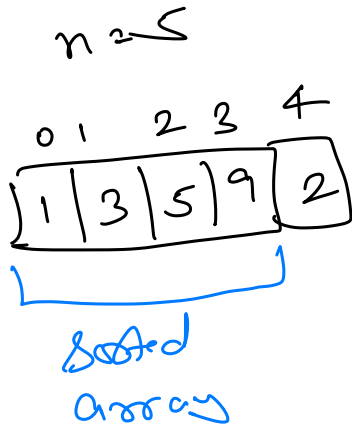
Insertion Sort

Given sorted elements, insert new element such that all elements remain sorted.





Look for the first element in sorted array that is smaller than the element to be inserted.
New element is added to the right of it.



sortedSize = 3

element To Insert = 3

$$i = (\text{sortedSize} - 1) = \cancel{2} \cancel{1} 0$$

if (element To Insert < arr[i])
 $\rightarrow \text{arr}[i+1] = \text{arr}[i]$

insertionSort(arr)

- sortedSize = 1

- while (sortedSize < n)

- elementToInsert = arr[sortedSize]

- i = sortedSize - 1

- while (i >= 0)

- if (elementToInsert < arr[i])

// elementToInsert is smaller than ith element => elementToInsert should be added to left of i

// Make space to left of i by moving ith element to right by one place.

- arr[i + 1] = arr[i]

else

- break

- i = i - 1

- arr[i + 1] = elementToInsert

- sortedSize = sortedSize + 1

Outer loop
(sortedSize)

1 →
2 →
⋮
(n-1) →

Inner loop
(i)

0 →
1 0 →
⋮
(n-2) ... 1 0 →

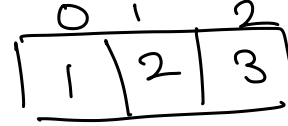
Total

1

2

⋮

→ (n-1)



n = 3

sortedSize = ~~1~~ ~~2~~ 3

elementToInsert = ~~2~~ 3

i → ~~0~~ 1

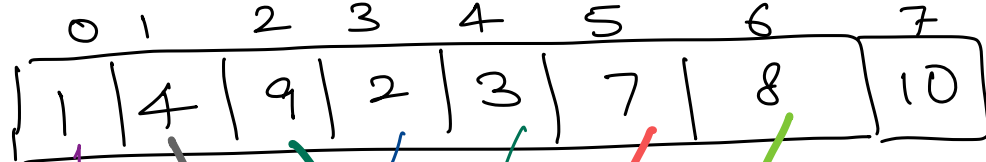
$$\text{Sum} = \frac{(n-1)(n)}{2}$$

$$\frac{(n-1)(n)}{2} = \frac{1}{2}(n^2 - n)$$

$$\text{Time complexity} = O(n^2)$$

$$\text{Space} = O(1)$$

$$\text{Best case time} = O(n)$$



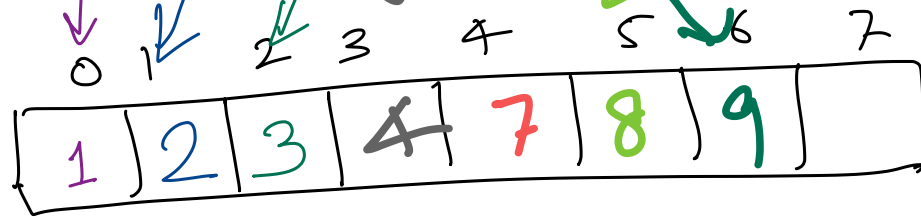
$n = 8$

start Pos 1 $\rightarrow 0$

end Pos 1 $\rightarrow 2$

start Pos 2 $\rightarrow 3$

end Pos 2 $\rightarrow 7$



$i1 \rightarrow 0$

$i2 \rightarrow 3$

$j \rightarrow 0$

~~3~~ ~~4~~ ~~5~~ ~~6~~

merged Array

Merge two sorted arrays.

Merge(arr, startPos1, endPos1, startPos2, endPos2)

- Set i1 to startPos1
- Set i2 to startPos2
- Set r to 0
- while ((i1 <= endPos1) and (i2 <= endPos2))
 - if (arr[i1] < arr[i2])
 - mergedArray[r] = arr[i1]
 - Increment i1 by 1
 - else
 - mergedArray[r] = arr[i2]
 - Increment i2 by 1
 - Increment r by 1

// Copy remaining elements from other sorted array into mergedArray

- while (i1 <= endPos1)
 - mergedArray[r] = arr[i1]
 - Increment i1 by 1
 - Increment r by 1
- while (i2 <= endPos2)
 - mergedArray[r] = arr[i2]
 - Increment i2 by 1
 - Increment r by 1

// Copy merged elements back to arr

- Set i to startPos1
- Set j to 0
- while (i < r)
 - arr[i] = mergedArray[j]
 - Increment i by 1
 - Increment j by 1
- Stop